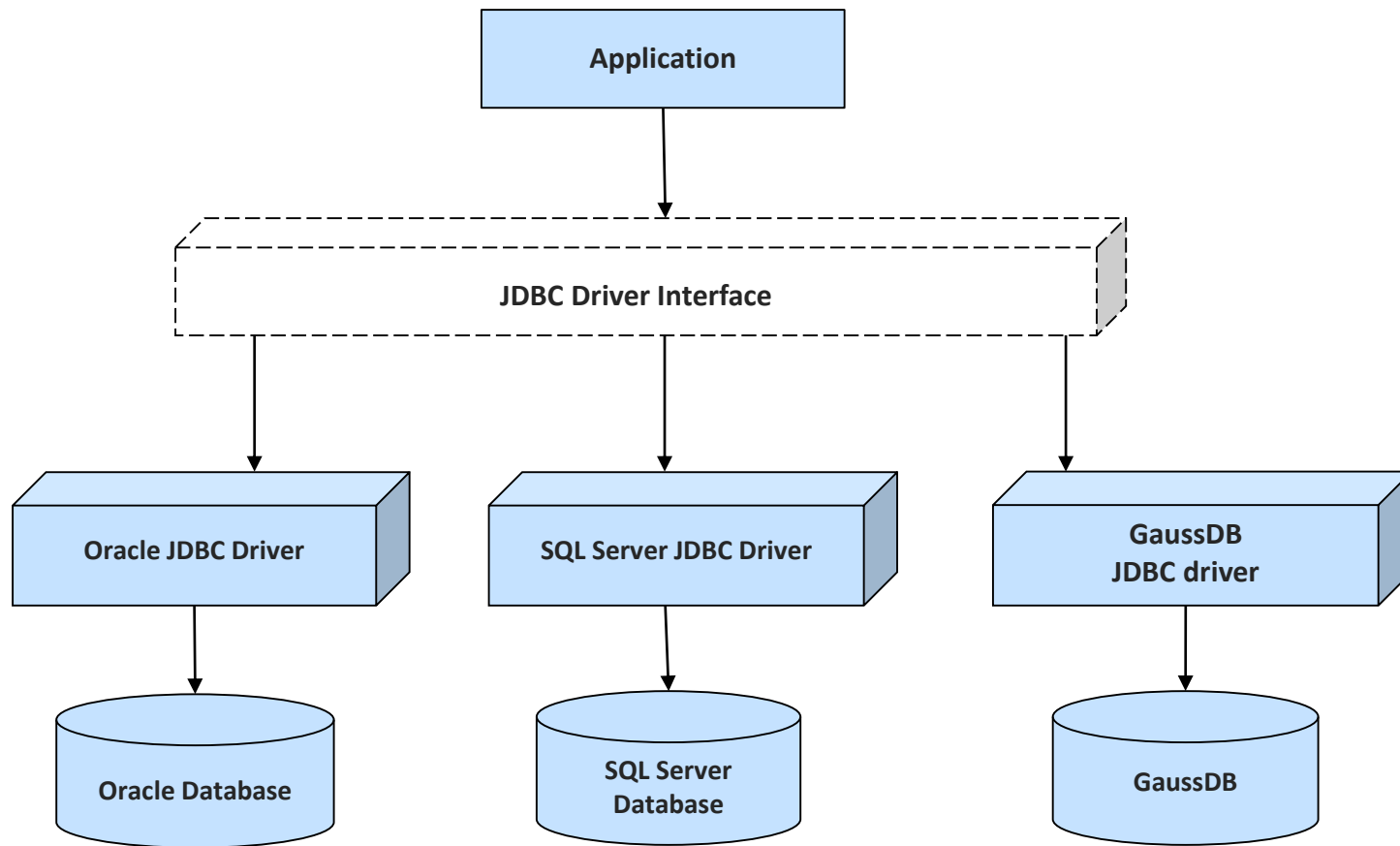




8.2 基于JDBC开发

JDBC系统结构介绍



注意：因JAVA本身具有良好的平台移植性，所以JDBC无平台限制。

常用数据类型对照表

数据库类型	JAVA数据类型	数据库类型	JAVA数据类型
VARCHAR(n) CHAR(n) NCHAR(n) text	java.lang.String	DOUBLE PRECISION, FLOAT8	double
"char"	byte	NUMERIC	java.math.BigDecimal
bytea	byte[]	DATE	java.sql.Timestamp
SMALLINT	short	TIME	java.sql.Time
INTEGER	int	TIMESTAMP	java.sql.Timestamp
BIGINT	long	CLOB	java.sql.Clob
FLOAT4, REAL	float	BLOB	java.sql.Blob
BOOLEAN	boolean		

JDBC包、驱动类和环境类

- **JDBC包**

- openGauss提供两种JDBCjar包：postgresql.jar和openGauss-jdbc-x.x.x.jar，两种jar包功能一致，仅仅是为了解决和PostgreSQL之间的JDBC驱动包名冲突。
- 在Linux服务器端源代码目录下执行build.sh，获得驱动jar包postgresql.jar和opengauss-jdbc-x.x.x.jar，包位置在源代码内output目录下。也可以从官网发布包中获取，包名为openGauss-x.x.x-JDBC.tar.gz。
- 驱动包与PostgreSQL保持兼容，其中类名、类结构与PostgreSQL驱动完全一致，曾经运行于PostgreSQL的应用程序可以直接移植到当前系统使用。

- **驱动类**

- openGauss在JDBC的使用上与PG的使用方法保持兼容。在同一个JVM虚拟机里加载PostgreSQL和openGauss的JDBC同名驱动时，可能存在org.postgresql.Driver驱动类冲突问题。

- **环境类**

- 客户端需配置JDK1.8。

使用用户名、密码作为参数建立连接

```
public static Connection getConnect(String username, String passwd)
{
    String driver = "org.opengauss.Driver"; //驱动类。
    String sourceURL = "jdbc:opengauss://10.10.0.13:8000/postgres"; //数据库连接描述符。
    Connection conn = null;
    try
    {
        Class.forName(driver); //加载驱动。
    }
    catch( Exception e )
    {
        e.printStackTrace();
        return null;
    }
    try
    {
        conn = DriverManager.getConnection(sourceURL, username, passwd); //创建连接。
        System.out.println("Connection succeed!");
    }
    catch(Exception e)
    {
        e.printStackTrace();
        return null;
    }
    return conn;
};
```

使用Properties对象作为参数建立连接

```
public static Connection getConnectUseProp(String username, String passwd)
{
    String driver = "org.opengauss.Driver"; //驱动类。
    String sourceURL = "jdbc:opengauss://10.10.0.13:8000/postgres?"; //数据库连接描述符。
    Connection conn = null;
    Properties info = new Properties();
    try
    {
        Class.forName(driver);    //加载驱动。
    }
    catch( Exception e )
    {
        e.printStackTrace();
        return null;
    }
    try
    {
        info.setProperty("user", username);
        info.setProperty("password", passwd);
        conn = DriverManager.getConnection(sourceURL, info); //创建连接。
        System.out.println("Connection succeed!");
    }
    catch(Exception e)
    {
        e.printStackTrace();
        return null;
    }
    return conn;
};
```

执行普通SQL语句

- 应用程序通过执行SQL语句来操作数据库的数据（不用传递参数的语句）。
- 调用Connection的createStatement方法创建语句对象。

```
Connection conn = DriverManager.getConnection("url","user","password");  
Statement stmt = conn.createStatement();
```

- ```
int rc = stmt.executeUpdate("CREATE TABLE customer_t1(c_customer_sk INTEGER, c_customer_name
VARCHAR(32));");
```

- ```
stmt.close();
```

执行预编译SQL语句

- 预编译语句是只编译和优化一次，然后通过设置不同的参数值多次使用。由于已经预先编译好，后续使用会减少执行时间。因此，如果多次执行一条语句，请选择使用预编译语句。
- 调用Connection的prepareStatement方法创建预编译语句对象。

```
PreparedStatement pstmt = con.prepareStatement("UPDATE customer_t1 SET c_customer_name = ? WHERE c_customer_sk = 1");
```

- 调用PreparedStatement的setShort设置参数
pstmt.setShort(1, (short)2);

处理结果集

- int rowcount = pstmt.executeUpdate();

- pstmt.close();
调用PreparedStatement的close方法关闭预编译语句对象。

调用存储过程

- 调用Connection的prepareCall方法创建调用语句对象。

```
Connection myConn = DriverManager.getConnection("url","user","password");  
CallableStatement cstmt = myConn.prepareCall("{? = CALL TESTPROC(?,?,?)}");
```

- 调用CallableStatement的setInt方法设置参数

```
cstmt.setInt(2, 50);  
cstmt.setInt(1, 20);
```

- cstmt.registerOutParameter(4, Types.INTEGER); //注册out类型的参数，类型为整型。

```
cstmt.execute();
```

- 调用CallableStatement的execute执行方法调用。

```
int out = cstmt.getInt(4); //获取out参数
```

- 调用CallableStatement的getInt方法获取输出参数。

```
cstmt.close();
```

- 调用CallableStatement的close方法关闭调用语句。

执行批处理

- 用一条预处理语句处理多条相似的数据，数据库只创建一次执行计划，节省了语句的编译和优化时间。
- 调用Connection的prepareStatement方法创建预编译语句对象。

```
Connection conn = DriverManager.getConnection("url","user","password");  
PreparedStatement pstmt = conn.prepareStatement("INSERT INTO customer_t1 VALUES (?)");
```

- ```
pstmt.setShort(1, (short)2);
pstmt.addBatch();
```

- ```
int[] rowcount = pstmt.executeBatch();
```

```
pstmt.close();
```

- 调用PreparedStatement的close方法关闭预编译语句对象。

处理结果集

- 不同类型的结果集有各自的应用场景，应用程序需要根据实际情况选择相应的结果集类型。在执行SQL语句过程中，都需要先创建相应的语句对象，而部分创建语句对象的方法提供了设置结果集类型的功能。

- `createStatement(int resultSetType, int resultSetConcurrency);`。

- `prepareStatement(String sql, int resultSetType, int resultSetConcurrency);`
● 创建一个PreparedStatement对象，该对象将生成具有给定类型和并发性的ResultSet对象。

- `prepareCall(String sql, int resultSetType, int resultSetConcurrency);`
● 创建一个CallableStatement对象，该对象将生成具有给定类型和并发性的ResultSet对象。

- `Connection conn = DriverManager.getConnection("url","user","password");`
`conn.close();`
● 在使用数据库连接完成相应的数据操作后，需要关闭数据库连接。